**Title**: **Design and Build a Data Model for Order and Shipment Reporting**.

**User Story**: As a Data Analyst, I want a well-structured analytical data model that accurately represents customer orders and shipment relationships, so that I can generate reliable insights on transactions, product performance, and delivery status without ambiguity or data duplication.

**Business Objective:** Enable accurate and scalable reporting by ensuring that transactional and shipment data are modeled with correct relationships, appropriate granularity, and historical tracking of key attributes.

## Scope:

- Design dimension and fact tables to support analytical reporting.
- Address data gaps identified in raw datasets.
- Ensure correct grain and relationships across datasets.
- Support accurate aggregation and delivery status tracking

## Technical Specifications

### Data Model Components:

- **Dimension Tables:**
  - Dim_Customer (SCD Type 2 with surrogate key).
  - Dim_Product
- **Fact Tables:**
  - **Fact_Order (order level)**
  - **Fact_product_order (order-product level).**
  - **Fact_Shipping (Mutable Fact table)**
- **Bridge Table:**
  - Order_Shipment_Bridge (order ↔ shipment mapping)

**Note**: Detailed column-level specifications, including data types, primary keys, foreign keys, and table structures, are documented in the attached Data Modeling document. The corresponding ERD diagram is also included for reference.

## Key Transformations:

- Generate surrogate key (customer_sk) for customer dimension.
- Implement SCD Type 2 logic for tracking customer attribute changes.
- Split the order dataset into fact order and fact product order tables based on order level and product-order level grain, respectively.
- Map order_id to shipment_id using bridge table.
- Remove customer_id dependency from shipping dataset.
- Implement update-in-place logic for the shipping table:
    - Initially insert shipment records with shipment_date populated and delivery_date as NULL and status as 'Pending'
    - Upon delivery, update the same record by populating delivery_date and updating the status to 'Delivered'.

## Final Relationship summary:

- Dim_Customer (1) ----- (M) Fact_Order
- Fact_Order (1) ------ (M) Fact_Product_Order
  Dim_Product (1) ------ (M) Fact_Product_Order
- Fact_Order (1) ------ (M) Order_shipment_bridge
- Fact_Shipping (1) ------ (M) Order_shipment_bridge
  Note: Bridge resolves MANY-to-MANY between Order and Shipping

## Business Rules:

- Each order must be associated with a valid customer record.
- Each order can contain one or multiple products.
- Each shipment can be associated with one or more orders.
- Each order can be associated with one or more shipments.
- If one of the shipments of the order is 'PENDING', then the overall status of the order is 'PENDING.'
- Each shipment is associated with a single status.
- Customer historical changes must be tracked using SCD Type 2
- Delivery status is also derived from:
    - delivery_date IS NULL → 'PENDING'
    - delivery_date IS NOT NULL → 'DELIVERED'.

## Acceptance Criteria:

- All tables are created with defined primary and foreign keys.
- Data model reflects the correct grain for each table.
- Bridge table correctly resolves many-to-many relationships.
- No duplicate records in Fact_Order (order_id unique).
- No duplicate records in Fact_product_order (order_id, product_id).
- All foreign keys correctly reference parent tables .
- customer_sk is populated for all orders.
- product_id is valid for all fact_product_order records.
- shipment_id is valid for all bridge records.
- delivery_date is NULL only for pending shipments

## QA Validation:

- Perform row count validation between source and target datasets.
- Validate referential integrity across all foreign keys.
- Test aggregation queries to ensure no duplication occurs.
- Validate SCD logic by simulating customer attribute changes

## Attachments:

- ERD Diagram
- Data model document: This has details of the table at the columnar level